Review: How to go beyond the black-box simulation barrier

Zihan Hu, Kai Su, Shiyu Zhao

Recap

- Zero-knowledge

  There exists a probabilistic polynomial-time simulator S such that for every polynomial-sized circuit family $\{V_N^*\}_{n\in\mathbb{N}}$ and every sequence $\{(x_n, y_n)\}_{n\in\mathbb{N}}$, where $x_n \in \{0,1\}^n \cap L$ and $(x_n, y_n) \in R$, we have

$$\left\{\text{view}_{V_n^*}\left\langle P\left(x_n, y_n\right), V_n^*\left(x_n\right)\right\rangle\right\}_{n\in\mathbb{N}} \approx_C \left\{S(V_n^*, x_n)\right\}$$

General Idea

Key point of simulator

- Challenge
  - convince verifier without witness
- Advantages
  - have access to random tape of verifier,

    predict next question to ask
  - retry when it fails (called *rewinding* technique)

Why non-black-box

blackbox = verifier as an oracle

non-black-box = code of verifier

- Control randomness?

  verifier can hide randomness into algorithm (PRF, hash function)

  - random, uncontrollable if it's black-box
  - Reverse-engineering is hard and code can be well-obfuscated

    still random, still uncontrollable even when it's non-black-box

- Rewinding(all zero-knowledge protocols)

  1. NO constant-round zero-knowledge proof in con-current composition
  2. NO constant-round zero-knowledge proof in strict polynomial time

Why non-black-box

- Rewinding

  "easy question": The questions simulator can output answer successfully

  Rewinding is just to **find** the "easy question"

- **Key idea!**

  What about modifying the definition of "easy question"?

  "easy question" = "what the verifier asks"

  So it can be trivially answered

- How?

FLS technique!

FLS Zero-Knowledge Argument

FLS technique

- Idea

  a framework that allows using **trapdoor information** to cheat the verifier while verifier **does not know it is cheated**

  - let **trapdoor information** to be the question verifier asks

    $\Rightarrow$ Knowing what the verifier asks is enough to convince/cheat the verifier

    $\Leftrightarrow$"easy question" is just "what the verifier asks" as desired

  - Role of description: to define the "easy question" to be what the verifier asks

FLS technique

- Idea
- Witness-indistinguishable

Let L = L(R) be some language and let (P, V) be an interactive argument for L. We say that (P, V) is witness-indistinguishable if for every polynomial-sized circuit family $\{V_n^*\}_{n\in\mathbb{N}}$ and every sequence $(x_n, y_n, y_n')_{n\in\mathbb{N}}$, where $x_n \in \{0,1\}_n$ and $(x_n, y_n), (x_n, y_n') \in R$, we have

$$\left\{\text{view}_{V_n^*}\left\langle P\left(x_n, y_n\right), V_n^*\left(x_n\right)\right\rangle\right\}_{n\in\mathbb{N}} \approx_C \left\{\text{view}_{V_n^*}\left\langle P\left(x_n, y_n'\right), V_n^*\left(x_n\right)\right\rangle\right\}_{n\in\mathbb{N}}$$

- Let the view of proving with witness for x and witness for trapdoor be indistinguishable $\Rightarrow$ verifier does not know it's cheated

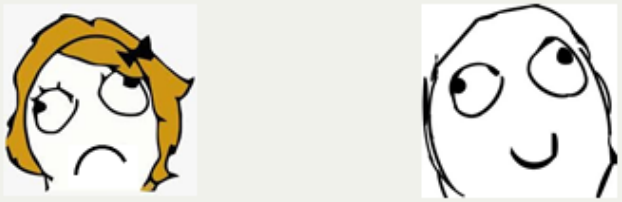
FLS-type zero-knowledge protocol

Public input: $x \in \{0,1\}^n$ (statement to be proved is "$x \in L$") **Prover's auxiliary input:** w (a witness that $x \in L$)	$w \downarrow \boxed{P}$ $\quad x \downarrow \boxed{V}$
Steps P,V1.x (Generation protocol): Prover and verifier engage in a *generation protocol* GENPROT . We denote the transcript of the execution by τ.	$1^n \downarrow \boxed{\text{GENPROT}} \downarrow \tau$
Steps P,V2.x (WI Proof): Prover proves to verifier using its auxiliary input w via a witness-indistinguishable (WI) proof/argument system that either $x \in L$ or $\tau \in \Lambda$, where Λ is a fixed language (which is part of the protocol's specification).[6]	$w \downarrow \quad x, \tau \downarrow$ $\boxed{\begin{array}{l} WI\text{-}proof \\ x \in L \\ \textbf{or } \tau \in \Lambda \end{array}} \downarrow 0/1$
The verifier accepts if the WI proof of the second stage is completed successfully (i.e., if the verifier algorithm for the WI proof accepts).	

Notice: In WI proof, $\langle x, \tau \rangle \in L'$ if either $x \in L$ or $\tau \in \Lambda$

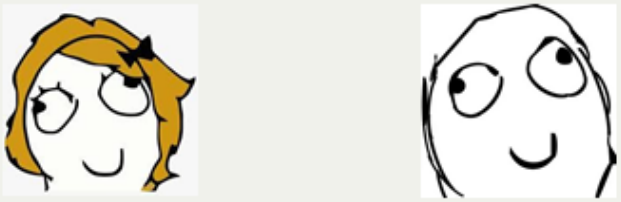
Stage 1: Alice and Bob play a game.
Bob wins in real execution but we make him lose in simulation.

Real Execution | **Simulation**



Bob wins.
$\tau \notin \Lambda$

$\approx_c$
(GenProt simulation)



Bob loses!
$\tau \in \Lambda$

Stage 2: Alice show $x \in L$ or $\tau \in \Lambda$.

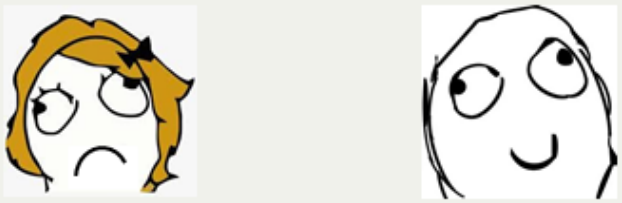
$x \in L$
and I have a witness.

$\approx_c$
(witness indistinguishable)

$\tau \in \Lambda$
and I have a witness.

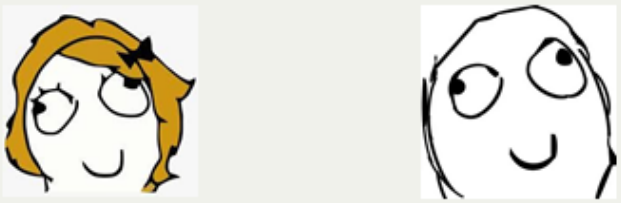
Stage 1: Alice and Bob play a game.
Bob wins in real execution but we make him lose in simulation.

Real Execution **Simulation**



Bob wins.
$\tau \notin \Lambda$

$\approx_c$
(GenProt simulation)



Bob loses!
$\tau \in \Lambda$

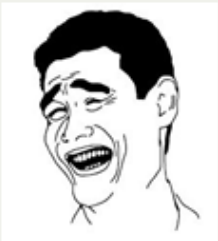
GenProt

1. Soundness: $\tau \notin \Lambda$ in real execution
2. Simulation:
   1. $\tau \in \Lambda$
   2. indistinguishable

Stage 1: Alice and Bob play a game.
Bob wins in real execution but we make him lose in simulation.

Game #1: Bob wins if he outputs some “random” string.

Real Execution



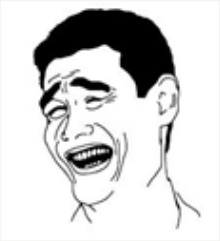
What’s on random tape?

Randomness

$\approx_c$

(PRG)

Simulation



What’s on random tape?

Pseudo-randomness

Stage 1: Alice and Bob play a game.
Bob wins in real execution but we make him lose in simulation.

Game #1: Bob wins if he outputs some "random" string.

Real Execution

Randomness

$\neq$

Simulation

Pseudo-randomness

Distinguishable via Kolmogorov Complexity
(The string is the output of a small TM)

Incompressible w.h.p.

- For all n and c,

$$\Pr_{x\in\{0,1\}^n}[K(x)\geq|x|-c]\geq 1-\frac{1}{2^c}$$

Proof credit: Theory of Computation Lecture 6

Compressible to $0.5n$

Machine M:
$u \leftarrow \{0,1\}^{0.1n}$
$s = PRG(u)$
Output $Bob(s)$

Uniform!

Game #1: Bob wins if he outputs some "random" string.

$$\text{Prover} \xleftarrow{r} \text{Verifier}$$

$$M : \text{a TM that outputs } r$$

$$\Lambda = \left\{ r : \exists M, |M| < \frac{|r|}{2}, M() = r \text{ in } |r|^{\log \log |r|} \right\}$$

Game #2: Bob wins if:

he has a different output from the program sent by Alice.

Real Execution

Some program M

Jp)fF3=@

...

$\approx_c$

(commitment)

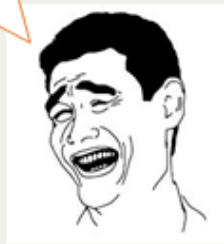
Simulation

123456

Bob's code along with his randomness

123456

As expected.



Game #2: Bob wins if:
he has a different output from the program sent by Alice.

$$\text{Prover} \xrightarrow{z} \text{Verifier}$$

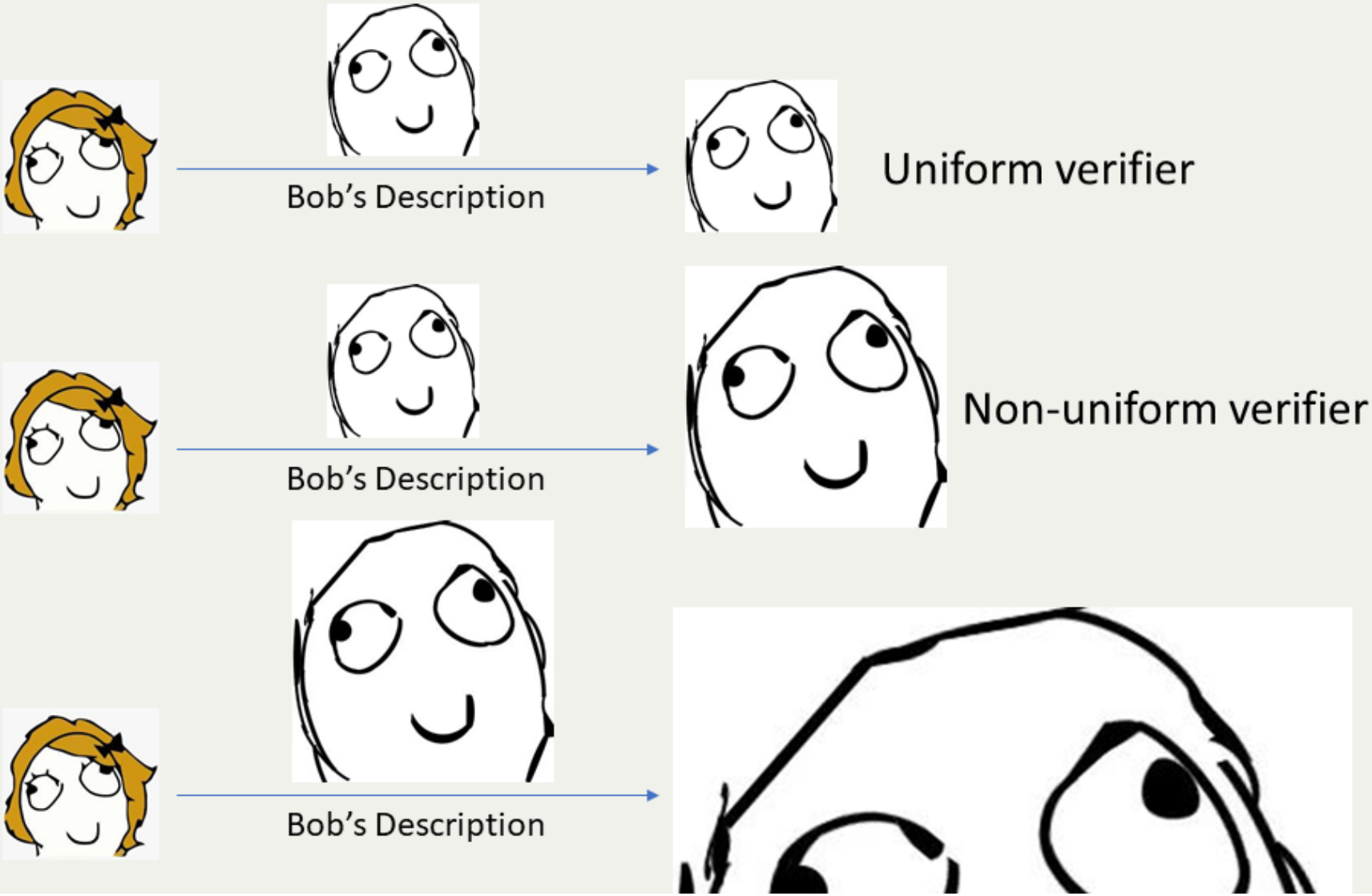
$$\text{Prover} \xleftarrow{r} \text{Verifier}$$

$$\Lambda = \left\{ (z, r) : \exists (s, \Pi), \mathrm{Com}(\Pi; s) = z, \Pi(z) = r \text{ in } |r|^{\log \log |r|} \right\}$$

Non-Uniform Verifier

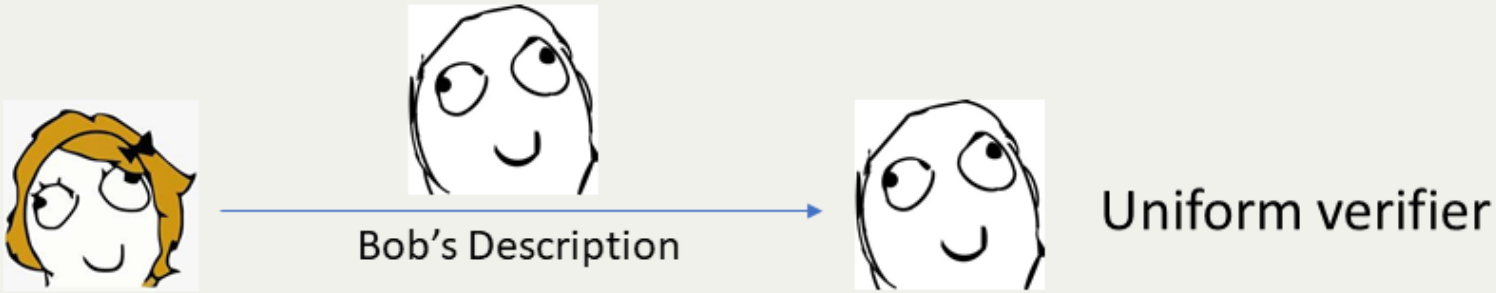
Game #2: Bob wins if:

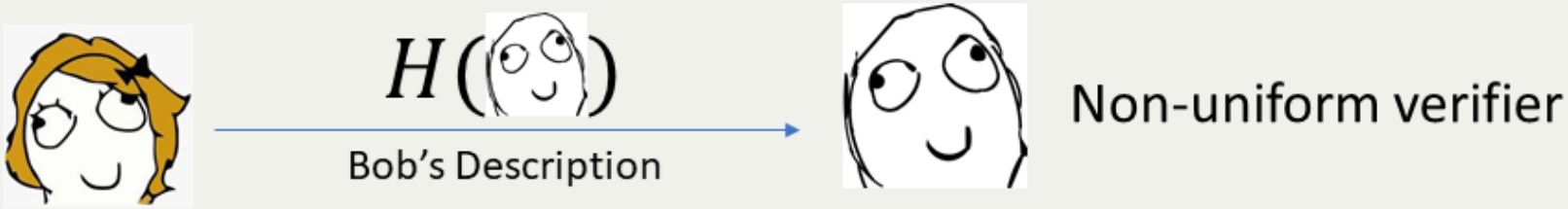
he has a different output from the program sent by Alice.



Game #2: Bob wins if:

he has a different output from the program sent by Alice.





Game #3: Bob wins if:

he has a different output from the program

whose hash is sent by Alice.

Game #3: Bob wins if:
he has a different output from the program
whose hash is sent by Alice.

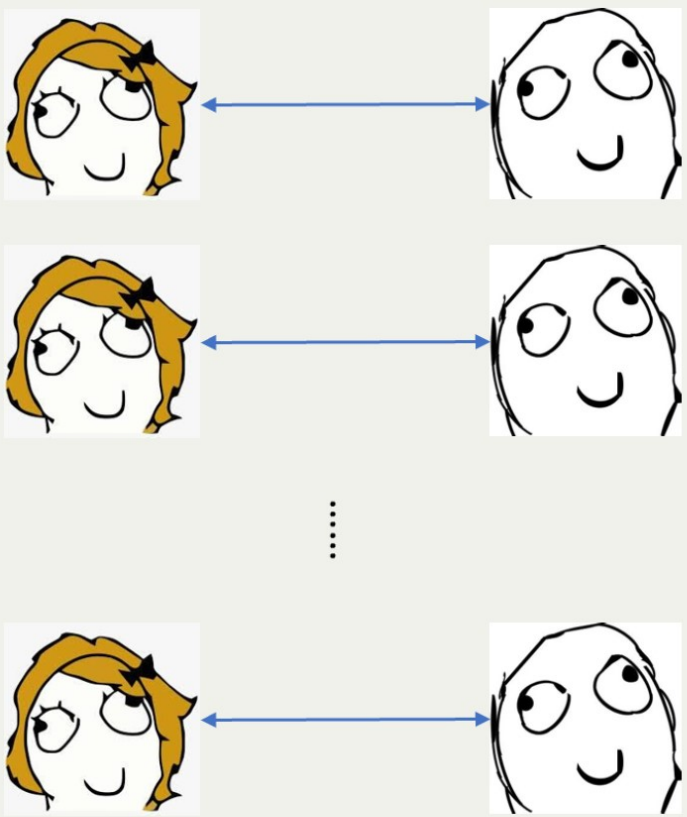
$$\text{Prover} \xleftarrow{h} \text{Verifier}$$

$$\text{Prover} \xrightarrow{z} \text{Verifier}$$

$$\text{Prover} \xleftarrow{r} \text{Verifier}$$

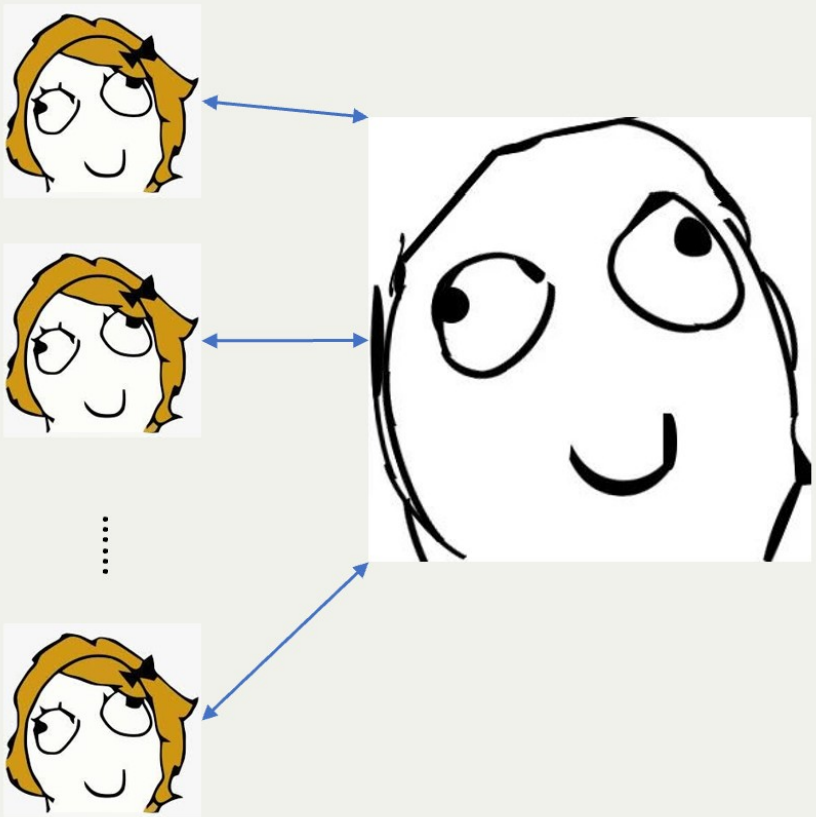
$$\Lambda = \left\{ (h, z, r) : \exists (s, \Pi), \mathrm{Com}(h(\Pi); s) = z, \Pi(z) = r \text{ in } |r|^{\log \log |r|} \right\}$$

Sometimes prover proves related statements simultaneously...



- t-times concurrent execution

  Run t **independent** copies of P with **one** verifier V on t instances. V chooses which prover to ask at each time and is given the response from the corresponding prover.

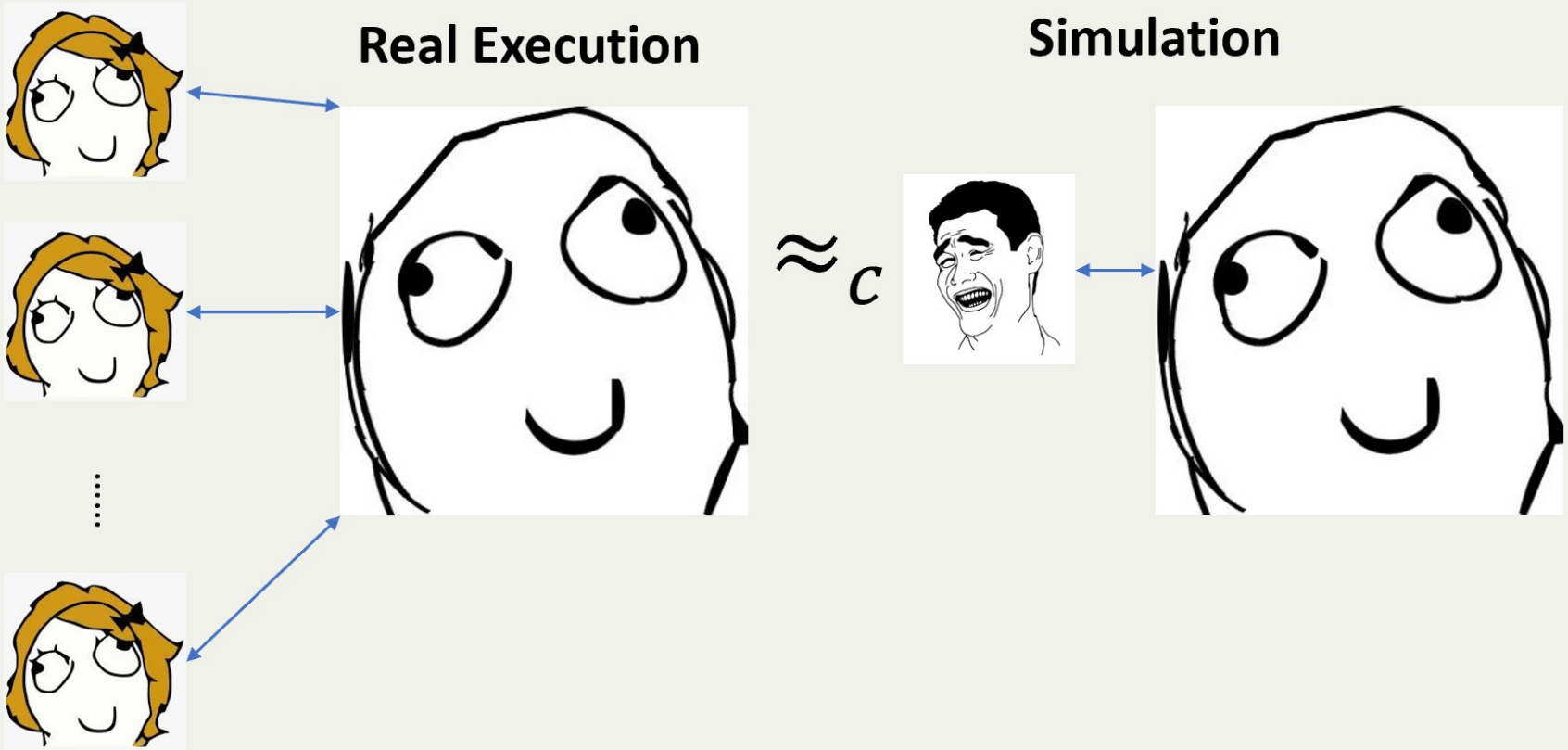


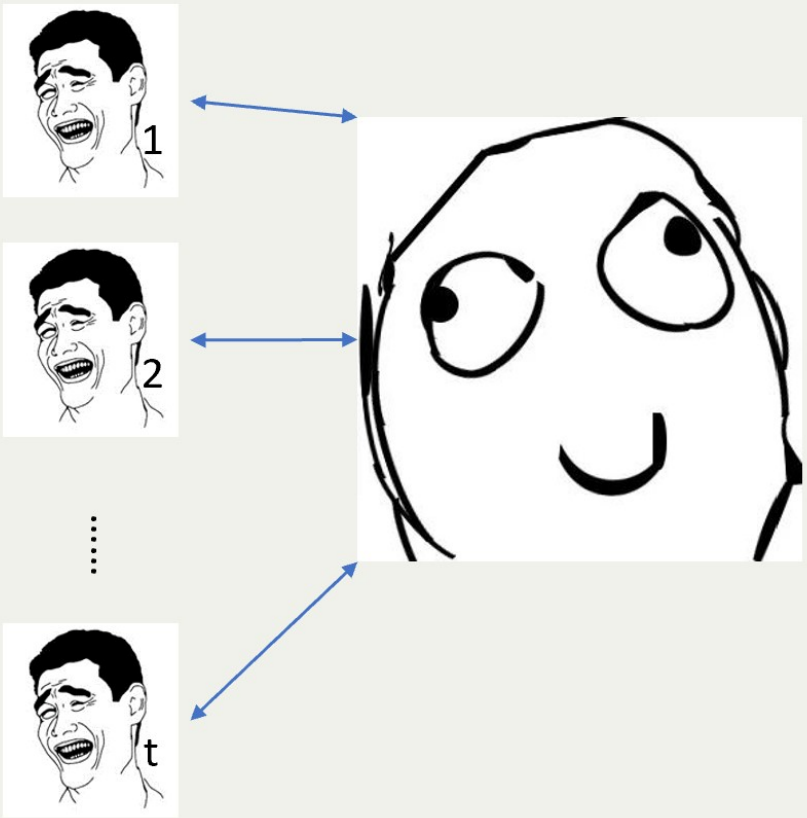
- bounded concurrent zero-knowledge

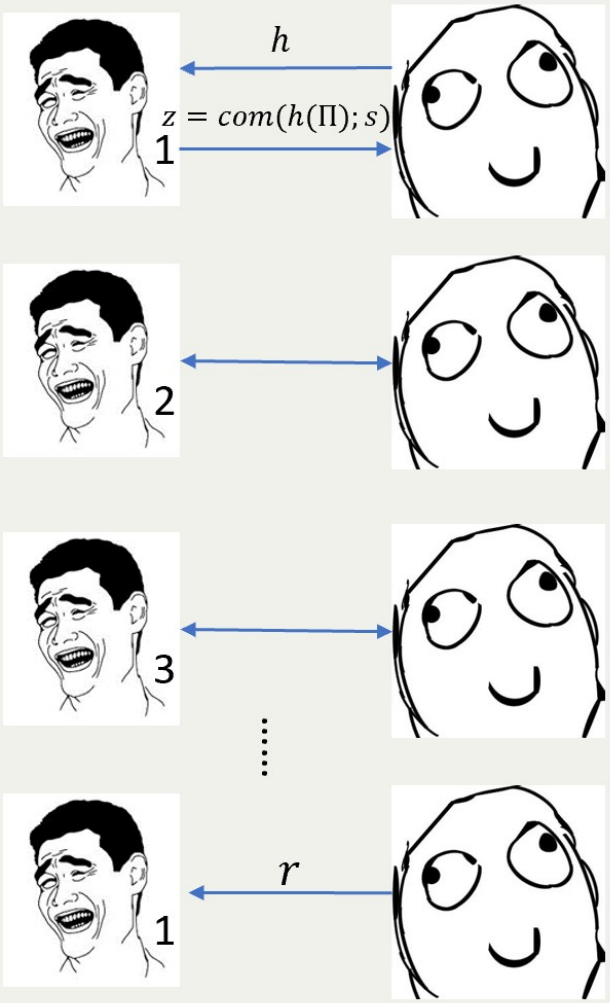
$\exists$ p.p.t. simulator S such that $\forall$p.p.t. V^* and every instance $\{(x_i, y_i)\}_{i=1}^n$ such that y_i is the witness for $x_i \in L$,

$\{$View of V in the real n-times concurrent execution$\} \approx_c S(V^*, x_1, x_2, \ldots, x_n)$

"bounded": ***Only*** for a ***fixed polynomial***-time concurrent execution, say for n.







$$r = \Pi(z)?$$

$$\text{Prover} \xleftarrow{h} \quad \text{Verifier}$$
$$\text{Prover} \xrightarrow{z} \quad \text{Verifier}$$
$$\text{Prover} \xleftarrow{r} \quad \text{Verifier}$$

Non-uniform case

$$\Lambda = \{(h, z, r) : \exists (s, \Pi), \mathrm{Com}(h(\Pi); s) = z, \Pi(z) = r \text{ in } |r|^{\log \log |r|}\}$$

$$\downarrow$$

Bounded concurrent zero-knowledge case

$$\Lambda = \{(h, z, r) : \exists (s, \Pi, y), \mathrm{Com}(h(\Pi); s) = z, \Pi(z, y) = r \text{ in } |r|^{\log \log |r|}\}$$

Proof Sketch

$$\Lambda = \{(h, z, r) | \exists (s, \Pi, y), \mathrm{Com}(h(\Pi); s) = z, \Pi(z, y) = r \text{ in } |r|^{\log \log |r|}\}$$

- Simulator: two parts
  - $S_{GenProt}$ generates the view of generation protocol (the first stage) and the trapdoor information (s, Π, y) along with (h, z, r)
  - S_{WI} generates the view of witness indistinguishable universal argument (the second stage)
- Completeness (trivial)
- Soundness

Proof Sketch (Cont'd)

$$\Lambda = \{(h, z, r) | \exists (s, \Pi, y), \mathrm{Com}(h(\Pi); s) = z, \Pi(z, y) = r \text{ in } |r|^{\log \log |r|}\}$$

- Simulator
- Completeness (trivial)
- Soundness

  If verifier will be convinced with non-negligible probability for random r,

  consider random $r \neq r'$,

  $\mathrm{Com}(h(\Pi); s) = z, \Pi(z, y) = r$, $\mathrm{Com}(h(\Pi'); s') = z, \Pi'(z, y') = r'$

  $\Rightarrow h(\Pi) = h(\Pi')$, if $|y| < \frac{|r|}{2}$, $\Pi \neq \Pi'$

Non-uniform case

$$\Lambda = \{(h, z, r) | \exists (s, \Pi), \mathrm{Com}(h(\Pi); s) = z, \Pi(z) = r \text{ in } |r|^{\log \log |r|}\}$$

$$\downarrow$$

Bounded concurrent zero-knowledge case

$$\Lambda = \{(h, z, r) | \exists (s, \Pi, y), |y| < \frac{|r|}{2}, \mathrm{Com}(h(\Pi); s) = z, \Pi(z, y) = r \text{ in } |r|^{\log \log |r|}\}$$

Summary

Assuming the existence of CRH (against some adversaries), there is a zero-knowledge argument system for NP satisfying the following properties:

- negligible soundness error
- for non-uniform V^*
- constant rounds and public coin
- bounded concurrent zero-knowledge
- strict polynomial time simulator

Subsequent work

why we need CRH against a super polynomial adversary?

Subsequent work

why we need CRH against a super polynomial adversary?

based on more standard assumption?

[BG01] Suppose CRH against ***polynomial***-sized adversary exists, universal argument and witness-indistinguishable universal argument exist. Furthermore, these proof systems are of ***public coin*** type and use a ***constant*** number of rounds.

Proof Sketch

- universal argument: via PCP
- + witness indistinguishable: via concurrent execution of "encrypt" version of universal argument + constant error soundness zero-knowledge proof
- how to use in this work:
  - hope that a random bit is different (via error correcting code)
  - hope that we can decommit to a single bit (via tree commitment)

Future directions

- Non-black-box proofs of security
- Black box reduction and non-black box reduction
- Arguments vs. proof
- Fiat-Shamir heuristic

Thank you!